Student Session Analytics Generation

Reflective, privacy-first session summaries for learners

1. Activity Overview & Objective

This activity generates **student-facing analytics** that summarize individual participation and engagement patterns during a session. Using attendance records and engagement scores, the system produces structured session summaries covering **attendance duration**, **engagement consistency**, and **notable activity trends**.

> **Reflective & Privacy-First Analytics Philosophy.** Student analytics are presented in a reflective format so learners can understand their participation without grades or punitive evaluations. Engagement timelines help students identify periods of high focus and distraction, encouraging self-improvement and accountability while strictly avoiding peer comparisons or public rankings.

Objectives

- Summarize each student's attendance and engagement for a session in a structured, reportable form.
- Expose chronological attention and emotion data so students can reflect on their own focus patterns.
- Support both dashboard visualization and downloadable post-session reports.
- Safeguard privacy: no cross-student percentiles, leaderboards, or rankings.

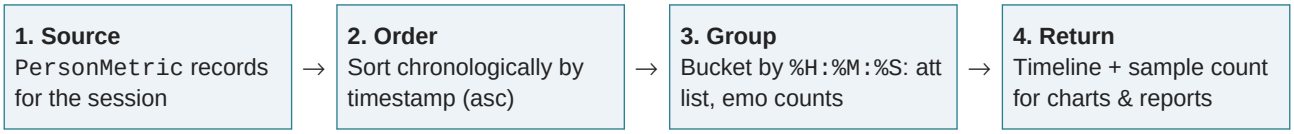
2. Student Session Summary Dimensions

To support dashboard visualization and downloadable report generation while safeguarding learner privacy, individual telemetry from `SessionPerson` and `PersonMetric` records is aggregated into four core analytical dimensions.

Analytics Dimension	Data Source & Aggregation Logic	Reflective Purpose for Learners
Attendance Duration	Cumulative active presence timestamps recorded in `SessionPerson` across the session lifecycle.	Transparent verification of session attendance and total active participation time.
Engagement Consistency	Temporal variance and stability of attention scores across sequential time buckets.	Helps students recognize how steadily they maintained focus across different learning modules.
Engagement Timelines	Chronological series (`%H:%M:%S`) of attention values (`att`) and affective states (`emo`).	Pinpoints exact windows of high focus versus distraction to encourage personal habit reflection.
Privacy-Preserving Scope	Isolated per-student metric queries with zero cross-student percentile or leaderboard ranking.	Fosters psychological safety and intrinsic self-improvement rather than competitive anxiety.

Table 1: The four analytical dimensions of a student session summary.

Analytics Generation Flow



3. Backend Implementation

The `AnalyticsService` class (`backend/core/analytics_service.py`) queries time-series telemetry from the database using SQLAlchemy, orders `PersonMetric` entries chronologically, and groups attention and emotion indicators into formatted timestamp buckets ready for frontend chart downsampling and student report rendering.

```
# File: backend/core/analytics_service.py
from sqlalchemy.orm import Session
from sqlalchemy import func
from .database import SessionLocal, Session as SessionModel, SessionPerson, PersonMetric

class AnalyticsService:
    def get_session_trends(self, session_id: int):
        db = SessionLocal()
        try:
            # Aggregate attention over time (minute buckets)
            # SQLite specific time grouping
            # For simplicity, we just return all metrics and let
            # frontend downsample or bin

            # Fetch last 1000 points to avoid overload
            metrics = db.query(PersonMetric)\
                .filter(PersonMetric.session_id == session_id)\
                .order_by(PersonMetric.timestamp.asc())\
                .all()

            # Group by timestamp (seconds)
            timeline = {}
            for m in metrics:
                ts_str = m.timestamp.strftime("%H:%M:%S")
                if ts_str not in timeline:
                    timeline[ts_str] = {'att': [], 'emo': {}}
                timeline[ts_str]['att'].append(m.attention_score)
                if m.emotion:
                    timeline[ts_str]['emo'][m.emotion] = (
                        timeline[ts_str]['emo'].get(m.emotion, 0) + 1
                    )

            return {
                "session_id": session_id,
                "timeline": timeline,
                "total_samples": len(metrics)
            }
        finally:
            db.close()
```

Output Structure

The method returns a dictionary with three keys. The example below is illustrative (values are made up) and shows the shape of the data.

```
{
  "session_id": 42,
  "timeline": {
    "10:15:03": { "att": [82, 76], "emo": { "neutral": 1, "happy": 1 } },
    "10:15:04": { "att": [79],     "emo": { "neutral": 1 } }
  },
  "total_samples": 3
}
```

Key	Type	Description
`session_id`	int	Identifier of the session being analyzed.
`timeline`	dict	Keyed by `HH:MM:SS`; each entry holds `att` (list of attention scores) and `emo` (emotion frequency map).
`total_samples`	int	Number of `PersonMetric` records retrieved for the session.

4. Key Engineering & Architectural Highlights

High-Resolution Temporal Grouping
By indexing `PersonMetric` records into `%H:%M:%S` timestamp dictionaries containing attention arrays (`att`) and emotion frequency maps (`emo`), the service preserves fine-grained session dynamics for flexible frontend binning.

Query Performance Guardrails
Bounding metric retrieval and ordering ascending by `PersonMetric.timestamp.asc()` prevents memory bottlenecks during extended lecture sessions while ensuring deterministic timeline rendering.

Non-Evaluative Self-Accountability
Structuring outputs around personal focus intervals and attendance consistency empowers students with actionable self-awareness suitable for private learner dashboards and post-session PDF summaries.

Implementation Notes

- The code comment mentions fetching the last 1000 points, but the query shown applies no `.limit()`, so all matching metrics are loaded. Adding a limit would make the guardrail effective.
- The method filters by `session_id` only, so the timeline aggregates every participant in the session. A per-student view, as described under Privacy-Preserving Scope, would need an additional filter on the student's record.
- Attendance duration and engagement consistency are defined as dimensions, but the code shown only produces the timeline. Those two values would be derived from `SessionPerson` timestamps and the variance of the attention series.